Bowen University Class Notes Demo

How the demo works and what every tag in the code does

1. What the demo is

A website where students, course reps and an admin log in to browse class notes. Everyone can see the course codes of **every** department. Every note link opens the same file, **class_note.pdf**. Logins, departments and courses are typed directly into the code, so no database or server is needed.

2. The files and how to run them

- **index.html** - the structure of the page (what is on it).
- **style.css** - the looks (colours, spacing, cards, buttons).
- **script.js** - the logic (login, tags, adding and deleting notes, requests, reports).
- **class_note.pdf** - the single sample note every link opens.
- Keep all four in the **same folder**, then double-click index.html. It works offline. index.html loads the other two through the <link> and <script> tags.

3. The main ideas

- **Public posting, department deleting.** When a rep posts a note, every department can see it. When a rep deletes a note, it is hidden only for students of the rep's own department. Other departments keep seeing it until their own reps delete it. So nobody can post and wipe something for everyone.
- **Course tags.** Every course has an owner (the department that made it) and tags (the departments it is related to). Students always see all courses, but can tick *Show only courses related to my department* to see just the ones tagged with their department.
- **General courses (my choice).** General courses like GST 111 belong to a department called General Studies with its own rep (repgst). They carry the tag "General", which counts as related to every department. So they appear in everyone's "only my department" view, and any rep can post notes to them.
- **Requests.** If a rep wants another department's course tagged for their own department, they send a request. The rep of the department that owns the course (or the admin) approves or declines it. Approving adds the requesting department to the course's tags.
- **Posted by.** Every note shows who posted it.
- **Reports.** Students can report a note. Reps see reports about courses related to their department. The admin sees all reports and can delete a note for every department.
- **Where reps can post.** A rep can add notes to any course related to their department: their own, General courses, and any course approved through a request.

4. How it flows

1. The page opens on the login card. The main page is hidden.
2. You type a username and password and press Login (or Enter). script.js checks them against the users list.

3. If wrong, an error shows. If right, the login card hides and the main page shows. Which panels appear depends on your role.
4. Students: filter by department, search, tick the department checkbox, open notes, and report notes.
5. Reps: also get panels to add a note and request a course, and an inbox for requests and reports.
6. Admin: gets the inbox and a Delete for all button on every note. Logout brings back the login card.

5. Demo logins

Role	Username	Password	Department	Can do
Student	student1	stud123	Information Technology	View all, filter, report
Student	student2	stud456	Cyber Security	Same
Student	student3	stud789	Software Engineering	Same
Course rep	repift	ift123	Information Technology	Add notes, delete for IFT only, send/approve requests, see reports
Course rep	repsen	sen123	Software Engineering	Same, for SEN
Course rep	repcyb	cyb123	Cyber Security	Same, for CYB
Course rep	repcsc	csc123	Computer Science	Same, for CSC
Course rep	repgst	gst123	General Studies	Same, for General courses
Admin	admin	admin123	Administration	Delete for all, approve any request, see all reports

Changes last only until the page is refreshed, because there is no database in this demo.

6. Try this walkthrough

1. Log in as **repift**, add a note to GST 111. It is public.
2. Log in as **student2** (Cyber Security). You see the note, with "Posted by IFT Course Rep". Tick the department checkbox: IFT courses vanish, GST and CYB stay.
3. Log in as **repift** and delete that note. Log in as **student1**: gone. Log in as **student2**: still there.
4. Log in as **repcyb**, request IFT 301. Log in as **repift** and approve it. Now student2's checkbox view includes IFT 301.
5. As **student2**, report a note. See the report as **repcyb**, then delete it for all as **admin**.

7. Every tag in index.html

A tag is a word in angle brackets that tells the browser what something is. Most come in pairs: an opening tag and a closing tag with a slash.

Tag	What it does in this demo
<!DOCTYPE html>	Tells the browser this is a modern HTML5 page. Always the first line.
<html lang="en">	Wraps the whole page. lang="en" says it is in English.
<head>	Holds information about the page that is not shown directly.
<meta charset="UTF-8">	Lets the page show all normal characters correctly.
<meta name="viewport" ...>	Makes the page fit phone screens.
<title>	The text on the browser tab.
<link rel="stylesheet" href="style.css">	Connects the page to the CSS file so the styles are applied.
<body>	Holds everything the user sees.
<!-- ... -->	A comment. The browser ignores it; it is a note for humans.
	A box that groups things: login page, main page, top bar, the three panels, filters and the course list.
<h1>, <h2>, <h3>	Headings from biggest to smaller: login card (h1), top bar (h2), panel titles (h3).
	A small piece of text inside a line, like the subtitle and the logged-in name.
<p>	A paragraph: subtitle, panel explanations, error and success messages.
<label>	The small title above an input, and the clickable text next to the checkbox.
<input>	A box to type in: username, password (type="password" hides letters), note title, search, and a tick box (type="checkbox").
<select>	A dropdown list: department filter, the rep's course choice, and the request course choice. script.js fills in the choices.
<button>	A clickable button: Login, Logout, Add Note, Send Request.
<script src="script.js">	Connects the page to the JavaScript file. Placed last so the page elements already exist when the code runs.

Tags that script.js creates while the page runs: **<option>** (one choice in a dropdown), **<ul>** and **<li>** (the list of notes), **<a href="class_note.pdf" target="_blank">** (a link that opens the PDF in a new tab), **<b>** (bold text for course codes), plus div, span, h3 and button for each course card, request and report.

Attributes used inside tags

Attribute	Meaning
id	A unique name so the JavaScript can find one element (e.g. id="loginPage").
class	A shared name so the CSS can style many elements the same way (e.g. class="course").

Attribute	Meaning
placeholder	Grey hint text inside an empty input.
type	The kind of input: "password" hides what you type, "checkbox" is a tick box.
value	The hidden value of a dropdown option (like "2-1" for department 2, course 1).
onclick	Runs a JavaScript function when clicked (e.g. onclick="login()").
oninput / onchange	Runs a function when the user types in a box, or ticks a box or changes a dropdown, so search and filters update live.
onkeydown	Runs code when a key is pressed. Used so Enter submits the login.
href / target	Where a link goes / "_blank" opens it in a new tab.
rel / src	rel="stylesheet" says the linked file is CSS; src names the JavaScript file to load.

8. style.css

Rule	What it styles
:root	Colour variables (blue, gold, background) used everywhere with var(--name).
body, *	Font and background; box-sizing keeps widths neat.
.hidden	display: none !important, which hides an element. !important makes sure it beats other display rules. The JavaScript adds and removes this class.
.login-wrap, .login-card, .logo	Gradient login background, white centred card, gold BU circle.
input, select, button	Size, border, focus glow and hover colour of typing boxes and buttons.
.topbar, .right, .light	The sticky blue top bar, the logged-in name area and the translucent Logout button.
.page, .filters, .check	Centres the content; lays out the dropdown, search and the department tick box.
.panel, .row, .item	The gold-edged panels, their row of controls, and one row in the requests or reports list.
#courseList	A responsive grid that fits as many course cards per row as the screen allows.
.course, .badge, .by	The white course card (lifts on hover), the tag badges, and the small "Posted by" text.
.del, .report, .ok-btn	Red Delete, amber Report and green Approve buttons.
display: flex / grid	Layout tools that arrange items in rows or grids.

9. script.js

Part	What it does
note() / course()	Helpers. note() makes a note (title, who posted it, and a hiddenFor list of departments it is deleted for). course() makes a course with three starting notes.
departments	All departments (including General Studies), each with its courses.
setupCourses()	Gives every course an owner and tags. General Studies courses get the tag "General"; others get their own department.
users	Every login: username, password, role (student, rep or admin) and department.
me, requests, reports	me is who is logged in (null if nobody). requests and reports are lists that hold course requests and reported notes.
setVisible()	Shows or hides an element by id, using the hidden class.
isRelated()	True if a course is tagged "General" or tagged with the logged-in user's department.
isHidden()	True if a note was deleted for the logged-in user's department.
canApprove()	True if the user is admin, or a rep of the department that owns the course.
login() / logout()	Check the details, show an error or the main page with the right panels for the role; logout returns to the login card.
fillFilter() / fillCourseSelect()	Fill the department dropdown, and fill a course dropdown with related or not-related courses.
showCourses()	Builds the course cards as text and puts them in the page. Applies the department filter, search and the "only my department" tick box, skips hidden notes, and adds Delete, Delete for all or Report buttons by role.
addNote()	Adds a public note, with the rep's name as the poster.
deleteNote()	After a confirm box, adds the rep's department to the note's hiddenFor list, so it disappears only for that department.
sendRequest() / answerRequest()	A rep asks for a course to be related to their department; the owner rep or admin approves (adds the department to the course's tags) or declines.
reportNote()	A student reports a note. Duplicate reports from the same student are blocked.
removeEverywhere(), deleteForAll(), deleteReported(), dismissReport()	Admin tools: delete a note for all departments (and clear its reports), or dismiss a report. Reps can dismiss reports too.
showInbox()	Builds the Requests and Reports lists for reps and admin.
getElementById / classList	getElementById finds an element by id; classList.add/remove adds or removes a class such as hidden.
for loops / if	for repeats code for each item in a list; if runs code only when a condition is true.

10. How to change things

- **Add a course:** add another course("CODE 101", "Title") inside a department in script.js.
- **Add a department:** copy one department line in departments and change the name and courses.
- **Add a login:** copy a line in users. Set role to student, rep or admin, and dept to the exact department name.
- **Make another course general:** put it under General Studies.
- **Change colours:** edit the variables in :root at the top of style.css.
- **Use a different note file:** replace class_note.pdf, or change the href in showCourses().

Note: passwords in script.js can be read by anyone who views the page source, so this is for demos only. A real version needs a server and database for logins and for saving notes, requests and reports.